Statistical Methods in AI - Assignment 4, Q1

Multi-Task CNN on Fashion-MNIST

Aks Kanodia
2023101020

November 4, 2025

---

1 Objective

The objective of this assignment is to implement a single multi-task Convolutional Neural Network (CNN) to perform two tasks simultaneously on the Fashion-MNIST dataset:

1. **Classification:** Classifying images into 10 clothing categories.
2. **Regression:** Predicting a continuous "ink" value, defined as the normalized average pixel intensity of the image.

The model is trained using a joint loss function, $\mathcal{L} = \lambda_1 \mathcal{L}_{classification} + \lambda_2 \mathcal{L}_{regression}$, where $\mathcal{L}_{classification}$ is Cross-Entropy loss and $\mathcal{L}_{regression}$ is Mean Squared Error (MSE). The goal is to explore the effect of the weighting factors λ_1 and λ_2 on the trade-off between the two tasks.

2 Methodology and Approach

2.1 Data Loading and Preprocessing

The Fashion-MNIST dataset was loaded using `torchvision`. The original training set of 60,000 images was split into a 90/10 ratio, yielding a training set of 54,000 images and a validation set of 6,000 images.

A custom PyTorch `Dataset` class, `FashionMNISTDataset`, was implemented as required. A key design choice was to calculate the `ink_target` (the average pixel value) from the **original, un-augmented** image. This ensures that the regression target remains a stable, ground-truth property of the image, even as the input image is augmented.

Two transformation pipelines were created:

- **Training:** Includes light augmentations (Random Rotation and Random Crop) to encourage generalization, followed by `ToTensor` and normalization.
- **Validation/Test:** Includes only `ToTensor` and normalization, ensuring the validation and test sets remain unaltered.

The dataset's global mean (0.2860) and std (0.3530) were used for normalization.

2.2 Model Architecture

A multi-task CNN was implemented as a `torch.nn.Module` subclass. The architecture consists of two main parts:

1. **Shared Backbone:** A shared base of three `Conv-BatchNorm-ReLU-Pool` blocks. This backbone processes the 1x28x28 input image and extracts features, downsampling it to a 128x3x3 feature map.

2. **Separate Heads:** The features from the backbone are flattened and fed into two independent heads:

    - **Classification Head:** A linear layer that outputs 10 logits for the classification task.
    - **Regression Head:** A linear layer that outputs a single scalar value for the ink prediction.

The model's `forward` method was designed to return both head outputs, as well as an optional dictionary of intermediate feature maps for later visualization.

2.3 Experimental Approach and Tuning

The experimental process was iterative and driven by key observations from Weights & Biases (wandb) logs.

1. Initial Exploration: Initial exploratory sweeps were run for 10-20 epochs, tuning learning rate, optimizer (Adam, AdamW, SGD), dropout, and λ values in a small range (e.g., 0.5 to 2.0). These runs revealed two things:

- A combination of high learning rate and high λ values could cause exploding gradients, resulting in `NaN` losses. This was solved by implementing gradient clipping (`clip_grad_norm_`) in the training step.
- The trade-off between classification and regression was minimal. All models achieved similar, high classification accuracy, regardless of the λ weights.

2. Key Insight: Loss Scale Disparity A deeper analysis of the logs revealed **why** the trade-off was not apparent: the two loss functions operate at vastly different scales.

- $\mathcal{L}_{CE}$ **(Classification):** Typically has values in the range of $\sim$0.2 - 1.0.
- $\mathcal{L}_{MSE}$ **(Regression):** Operates on normalized ink values, resulting in very small losses (e.g., $(0.5 - 0.45)^2 = 0.0025$).

Because the CE loss was naturally 100-1000x larger than the MSE loss, it completely dominated the total loss and the gradient updates. The model was, in effect, **always** prioritizing classification irrespective of how well it was learning the ink targets.

3. Final Targeted Experiment Armed with this hypothesis, a final, targeted grid search was designed to test it. We fixed $\lambda_1 = 1.0$ and varied λ_2 across a large range to artificially compensate for the scale difference. The final 3-run experiment used the best-found hyperparameters and tested:

- **Case 1 (Classification-Dominated):** $\lambda_2 = 1.0$

- **Case 2 (Balanced):** $\lambda_2 = 100.0$
- **Case 3 (Regression-Dominated):** $\lambda_2 = 1000.0$

All runs used the Adam optimizer, a learning rate of 1e-3, a dropout of 0.3, batch size of 128, and were run for 25 epochs.

3 Results and Analysis

3.1 Hyperparameter Tuning and Model Selection

The final 3-run sweep successfully demonstrated the performance trade-off. The key results on the test set are summarized in Table 1.

Table 1: Final Model Performance on Test Set

Model Type	λ_1	λ_2	**Test Accuracy (%)**	**Test RMSE**	**Test MAE**
Best Classifier	1.0	1.0	**91.56**	0.0363	0.0277
Balanced	1.0	100.0	90.63	0.0178	0.0146
Best Regressor	1.0	1000.0	88.53	**0.0082**	**0.0063**

As hypothesized, the model's performance was directly tied to the re-scaled loss.

- The **Best Classifier** ($\lambda_2 = 1.0$) achieved the highest accuracy (91.56%) but the worst regression performance.
- The **Best Regressor** ($\lambda_2 = 1000.0$) achieved a significantly lower RMSE (0.0082), but this came at a cost of 3% in classification accuracy.
- The **Balanced** model ($\lambda_2 = 100.0$) provided a strong compromise between the two objectives.

Figure 1 shows the validation accuracy and validation RMSE curves from these three runs, clearly illustrating the trade-off.

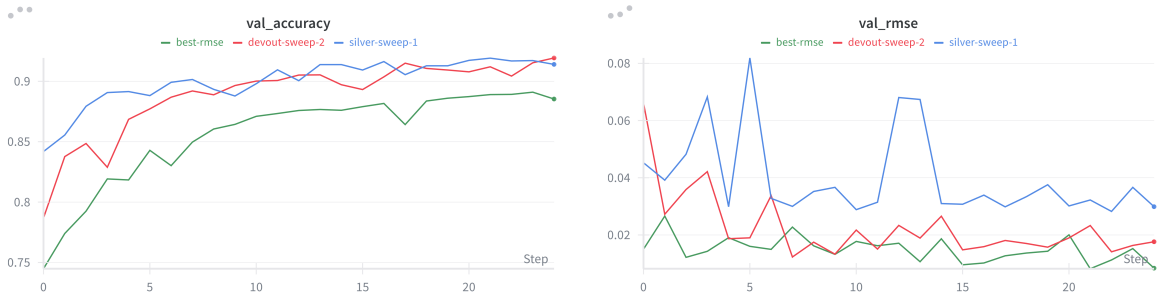


Figure 1: Validation Accuracy (left) and Validation RMSE (right) from the final 3-run sweep. The run with $\lambda_2 = 1.0$ (blue) achieves highest accuracy, while the run with $\lambda_2 = 1000.0$ (green) achieves lowest RMSE, but its accuracy suffers.

3.2 Feature Map Visualization

To understand what the model learned, the intermediate feature maps from the "Best Classifier" model were extracted for a test image (an Ankle Boot). The visualizations show a clear hierarchy of feature extraction.

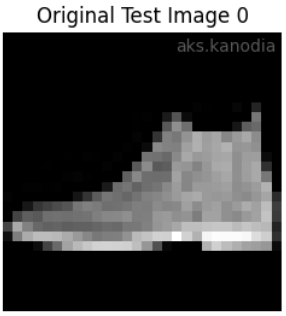


Figure 2: Original Test Image (Ankle Boot, Class 9)

- **Block 1 (Figure 3):** The maps are 14x14 and act as low-level feature detectors. We can clearly see filters that activate on simple horizontal and vertical edges, gradients, and basic textures, highlighting the object's silhouette.
- **Block 2 (Figure 4):** At 7x7, these maps combine the simple edges from Block 1 into more complex patterns and parts. The features are more abstract, learning to detect specific "parts" of the boot, like the curve of the heel or the flat line of the sole.
- **Block 3 (Figure 5):** At 3x3, the maps are highly abstract and sparse. They no longer resemble the input image. These filters act as high-level concept detectors. A single filter may activate when it "sees" the combination of features that define a "heel" or "toe," which is then fed to the final classification head.

This hierarchy allows the model to build a complex understanding of the image, which is beneficial for both identifying the class and estimating the total ink density. More such feature maps can be found in the jupyter notebook for this question, including feature maps for other images as well as the "best regressor" model.

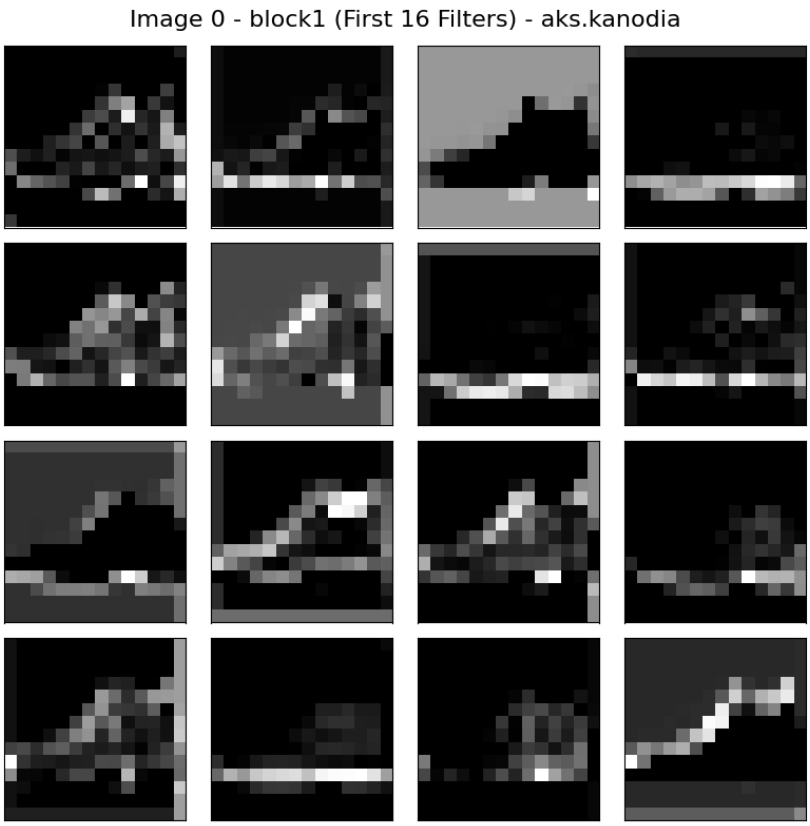


Figure 3: Feature Maps from Block 1 (Low-Level Edge Detectors)

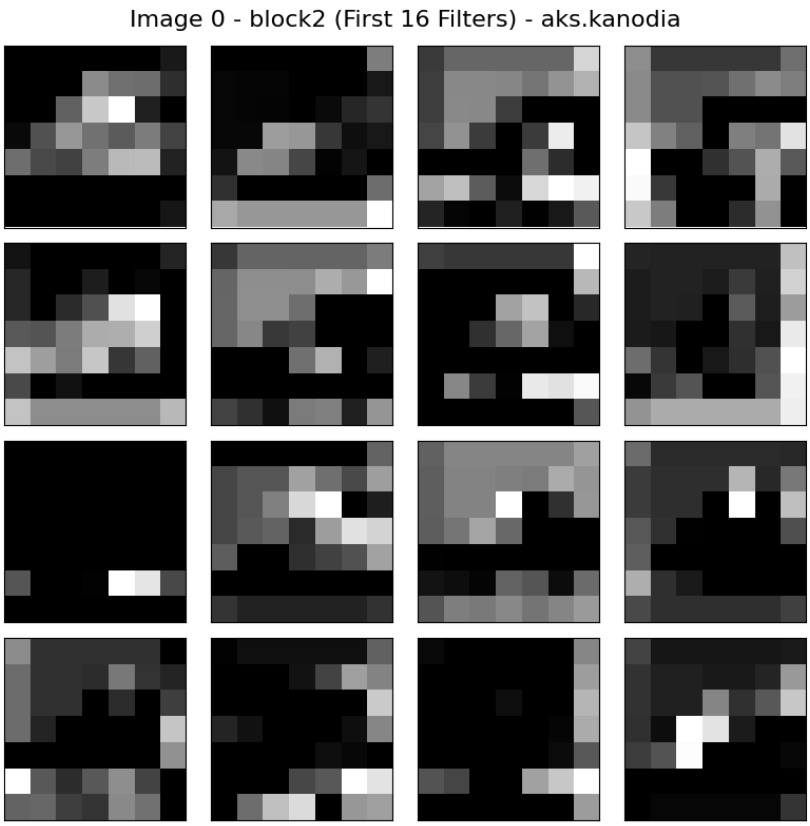


Figure 4: Feature Maps from Block 2 (Complex Patterns / Parts)

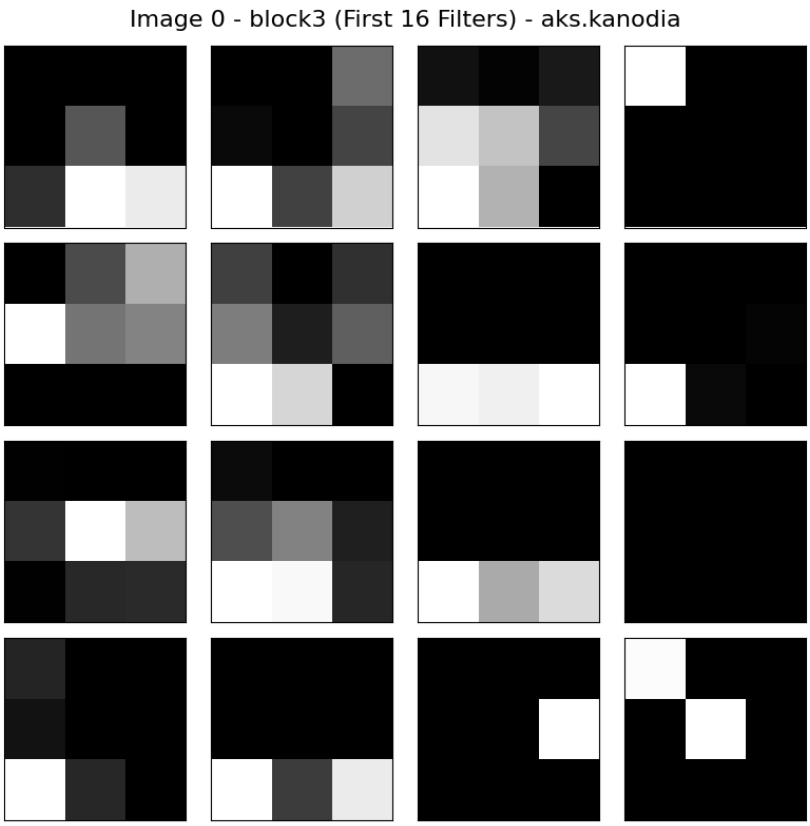


Figure 5: Feature Maps from Block 3 (High-Level Abstract Concepts)

4 Conclusion

This assignment successfully demonstrated the implementation of a multi-task CNN for joint classification and regression. The most critical finding was not the hyperparameters themselves, but the identification of the vast difference in scale between the Cross-Entropy and MSE loss functions. Initial experiments failed to show a trade-off because the classification loss dominated all training.

By forming a hypothesis about this disparity and designing a targeted experiment to compensate for it (by scaling λ_2 by 1000x), we successfully forced the model to prioritize regression, which yielded a significantly lower RMSE at the measurable cost of classification accuracy. This confirms the trade-off and highlights the importance of understanding and balancing loss scales in multi-task learning.

5 Deliverables

5.1 Weights & Biases Project

All experiments, configurations, and results (along with additional visualizations such as confusion matrices and example predictions) were logged to a public Weights & Biases project, which can be accessed at the following link:
`https://wandb.ai/kanodiaaks-iiit-hyderabad/smai-a4-q1/`